

Lecture 22

Zero-Knowledge Proofs: Sigma Protocols, Fiat–Shamir, SNARKs and STARKs

SEC 595: Encrypted Computing

Roadmap

A zero-knowledge proof convinces a verifier that a statement is true while revealing nothing else. Section 1 defines proof systems and their three properties, together with the simulator and the extractor. Section 2 gives the canonical example, Schnorr’s protocol, and Section 3 the commitments it is built from. Section 4 removes interaction with the Fiat–Shamir transform. Sections 5 to 7 scale up from “I know a discrete logarithm” to “I ran this program”: the computation is turned into a polynomial identity, the polynomial is committed to with KZG or with FRI, and the two choices give SNARKs and STARKs.

1. Proof Systems

1.1 Statements and Witnesses

Definition 1 (Relation and language). A **relation** $\mathcal{R}$ is a set of pairs (x, w) , where x is the **statement** and w the **witness**, such that membership $(x, w) \in \mathcal{R}$ is checkable in polynomial time. Its **language** is $L_{\mathcal{R}} = \{x : \exists w, (x, w) \in \mathcal{R}\}$.

The **prover** P holds x and w ; the **verifier** V holds only x , runs in polynomial time, and accepts or rejects. Two relations appear in this lecture:

$$\begin{aligned}\mathcal{R}_{\text{DL}} & ((\mathbb{G}, g, h), x) \text{ with } h = g^x; \\ \mathcal{R}_{\text{exec}} & ((\text{program}, \text{input}, \text{output}), \text{execution trace}).\end{aligned}$$

1.2 The Three Properties

Definition 2 (Interactive proof system). (P, V) is a proof system for $\mathcal{R}$ with **soundness error** ε if:

1. **Completeness.** If $(x, w) \in \mathcal{R}$, V accepts the interaction with $P(x, w)$.
2. **Soundness.** If $x \notin L_{\mathcal{R}}$, every cheating prover P^* is accepted with probability at most ε .
3. **Zero knowledge.** There is an efficient **simulator** $\mathcal{S}$ that, given x alone, outputs a distribution indistinguishable from the verifier’s view of a real interaction.

Completeness protects the honest prover, soundness protects the verifier, and zero knowledge protects the prover against a verifier that tries to learn.

If soundness holds against *all* provers, the system is a **proof**; if it holds only against efficient provers, it is an **argument**. Every succinct system in this lecture is an argument: the “AR” in SNARK and STARK stands for *argument*.

1.3 The Simulator

Zero knowledge is the simulation paradigm of Lecture 21, with the simulator given only the statement. The quantifiers are what matter: *for every* verifier V^* *there exists* a simulator whose output, from x alone, is indistinguishable from V^* 's real view. Whatever V^* computes from the transcript it could have computed without the prover, so the transcript carried nothing else.

Grade	Real and simulated views are
perfect	identically distributed
statistical	at negligible statistical distance
computational	indistinguishable to efficient algorithms (Lecture 1)

1.4 Proofs of Knowledge

In a cyclic group $\mathbb{G} = \langle g \rangle$ every h is a power of g , so $L_{\mathcal{R}_{DL}}$ is all of $\mathbb{G}$ and soundness is vacuous: there is no false statement. What we want is that the prover *knows* x .

Definition 3 (Proof of knowledge). (P, V) is a **proof of knowledge** if there is an efficient **extractor** $\mathcal{E}$ which, given rewinding access to any P^* that convinces V with non-negligible probability, outputs a witness w with $(x, w) \in \mathcal{R}$.

Rewinding means rerunning P^* from a saved state with a different challenge. The two devices pull in opposite directions: the simulator produces a transcript *without* the witness; the extractor produces the witness *from* transcripts.

1.5 Honest-Verifier Zero Knowledge

Definition 4 (HVZK and full ZK). A protocol is **honest-verifier zero knowledge** (HVZK) if $\mathcal{S}$ need only reproduce the view of a verifier that follows the protocol, so that its challenge is uniform and independent of the prover's first message. It is **(fully) zero knowledge** if $\mathcal{S}$ succeeds against every efficient V^* , including one that chooses its challenge as a function of what it saw.

The Fiat–Shamir transform (Section 4) makes the challenge a hash, which is modelled as uniform and independent. That is why HVZK suffices in practice.

2. Sigma Protocols and Schnorr

2.1 Sigma Protocols

Definition 5 (Sigma protocol). A **Sigma protocol** is a three-move protocol

$$\begin{array}{ll}
 P \longrightarrow V & : \text{commitment } a \\
 V \longrightarrow P & : \text{challenge } c \leftarrow \mathcal{C}, \text{ uniform} \\
 P \longrightarrow V & : \text{response } z
 \end{array}$$

satisfying completeness, HVZK, and **special soundness**: from accepting transcripts (a, c, z) and (a, c', z') with the same a and $c \neq c'$, a witness can be computed efficiently.

The protocol is **public coin**: the verifier sends only randomness. Special soundness gives an extractor at once: rewind the prover after a and ask a second challenge.

2.2 Schnorr's Protocol

Definition 6 (Schnorr). Let $\mathbb{G} = \langle g \rangle$ have prime order q ; the statement is $h = g^x$ and the witness is $x \in \mathbb{Z}_q$.

1. P samples $k \leftarrow \mathbb{Z}_q$ and sends $a = g^k$.
2. V samples $c \leftarrow \mathbb{Z}_q$ and sends it.
3. P sends $z = k + cx \pmod{q}$.
4. V accepts iff $g^z = a \cdot h^c$.

Completeness: $g^z = g^k(g^x)^c = ah^c$; exponents live modulo $q = \text{ord}(g)$. The nonce k acts as a one-time pad: $z = k + cx$ is uniform on $\mathbb{Z}_q$ and on its own reveals nothing about x .

Proposition 1 (Special soundness). *From accepting (a, c, z) and (a, c', z') with $c \neq c'$,*

$$x = (z - z')(c - c')^{-1} \pmod{q}.$$

Proof. Dividing $g^z = ah^c$ by $g^{z'} = ah^{c'}$ gives $g^{z-z'} = h^{c-c'}$; since q is prime, $c - c'$ is invertible modulo q . $\square$

So a prover that does not know x can answer at most one challenge per commitment, and its soundness error is $1/|\mathcal{C}|$. With $|\mathcal{C}| = 2^{128}$ one round suffices.

Proposition 2 (Perfect HVZK). *Given h and c , the simulator samples $z \leftarrow \mathbb{Z}_q$ and sets $a = g^z h^{-c}$. Its output is distributed exactly as a real transcript.*

Proof. In both, z is uniform on $\mathbb{Z}_q$ and independent of c , and a is fixed by the verification equation $g^z = ah^c$. $\square$

The simulator chose z *first* and solved for a . A real prover cannot: it must send a before it sees c . The order of moves is what separates a proof from a forgery.

2.3 Worked Instance

Example 1 (A Schnorr transcript). Let $\mathbb{G} = \langle 2 \rangle \subseteq \mathbb{F}_{23}^\times$. The powers $2^1, \dots, 2^{11}$ modulo 23 are

$$2, 4, 8, 16, 9, 18, 13, 3, 6, 12, 1,$$

so $\text{ord}(2) = 11$ and $\mathbb{G} = \{1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18\}$. Exponents live in $\mathbb{Z}_{11}$, group elements in $\mathbb{Z}_{23}$. With witness $x = 6$, the statement is $h = 2^6 = 64 \equiv 18 \pmod{23}$.

Run with $k = 4$, $c = 7$: $a = 2^4 = 16$ and $z = 4 + 7 \cdot 6 = 46 \equiv 2 \pmod{11}$. Check: $g^z = 2^2 = 4$, and $h^c = 18^7 \equiv 6$, so $ah^c = 16 \cdot 6 = 96 \equiv 4$. $\checkmark$

Example 2 (Rewind and simulate). Same instance.

Rewind. Keep $a = 16$ and ask $c' = 2$: $z' = 4 + 2 \cdot 6 = 16 \equiv 5 \pmod{11}$. Check: $2^5 = 32 \equiv 9$, and $ah^2 = 16 \cdot 2 = 32 \equiv 9$. $\checkmark$ The extractor computes $(c - c')^{-1} = 5^{-1} \equiv 9 \pmod{11}$ (since $5 \cdot 9 = 45 \equiv 1$) and

$$x = (z - z') \cdot 9 = (2 - 5) \cdot 9 = -27 \equiv 6 \pmod{11}. \checkmark$$

Simulate. With no witness, fix $c = 3$ and take $z = 10$. Then $h^3 = 18^3 \equiv 13$, $13^{-1} \equiv 16 \pmod{23}$ and $g^{10} \equiv 12$, so $a = 12 \cdot 16 = 192 \equiv 8$. The transcript $(8, 3, 10)$ verifies: $ah^c = 8 \cdot 13 = 104 \equiv 12 = g^z$. ✓

Soundness error. One round in this group has error $1/|\mathcal{C}| = 1/11 \approx 9\%$.

3. Commitments

3.1 Hiding and Binding

Definition 7 (Commitment scheme). $\text{Com}(m; r) \rightarrow c$ commits to m with randomness r ; opening reveals (m, r) and the receiver recomputes. It is **hiding** if c reveals nothing about m , and **binding** if the committer cannot open c to two different messages.

Proposition 3. *No commitment scheme is both perfectly hiding and perfectly binding.*

Proof. If c has the same distribution for every m , then for any m' some r' opens c to m' , and an unbounded committer finds it. □

Scheme	Hiding	Binding
Pedersen $g^m h^r$	perfect	computational (DLog)
Hash $H(m \parallel r)$	computational	computational (collision resistance)

3.2 Pedersen Commitments

Definition 8 (Pedersen). Let $\mathbb{G}$ have prime order q and generators g, h with $\log_g h$ unknown to the committer. For $m, r \in \mathbb{Z}_q$, $\text{Com}(m; r) = g^m h^r$.

Proposition 4. *Pedersen is perfectly hiding, and binding under the discrete logarithm assumption (Lecture 1).*

Proof. *Hiding:* for fixed m , h^r is uniform on $\mathbb{G}$, so $g^m h^r$ is uniform for every m . *Binding:* $g^m h^r = g^{m'} h^{r'}$ with $m \neq m'$ gives $\log_g h = (m - m')(r' - r)^{-1} \pmod{q}$. □

Proposition 5 (Homomorphism). $\text{Com}(m_1; r_1) \cdot \text{Com}(m_2; r_2) = \text{Com}(m_1 + m_2; r_1 + r_2)$, both sums modulo q .

So a prover can prove linear relations among committed values without opening them. To show $m_3 = m_1 + m_2$, note that $\text{Com}(m_1; r_1) \text{Com}(m_2; r_2)$ and $\text{Com}(m_3; r_3)$ differ by a commitment to zero, h^r , and give a Schnorr proof of knowledge of r . Schnorr's $a = g^k$ plays the same role as a commitment: it freezes k before the challenge.

Example 3 (Pedersen in $\langle 2 \rangle \subseteq \mathbb{F}_{23}^\times$). Take $g = 2$ and $h = g^7 = 13$. Group elements are reduced modulo 23, exponents modulo $q = 11$.

$$\text{Com}(5; 3) = 2^5 \cdot 13^3 = 9 \cdot 12 = 108 \equiv 16, \quad \text{Com}(4; 9) = 2^4 \cdot 13^9 = 16 \cdot 3 = 48 \equiv 2.$$

Their product is $16 \cdot 2 = 32 \equiv 9$. The openings add to $(5 + 4, 3 + 9) = (9, 12) \equiv (9, 1) \pmod{11}$, and $\text{Com}(9; 1) = 2^9 \cdot 13 = 6 \cdot 13 = 78 \equiv 9$. ✓

Binding. $\text{Com}(3; 4) = 8 \cdot 18 = 144 \equiv 6$ and $\text{Com}(0; 6) = 13^6 \equiv 6$. The binding argument extracts $\log_g h = (3 - 0)(6 - 4)^{-1} = 3 \cdot 6 = 18 \equiv 7 \pmod{11}$. ✓

4. Fiat–Shamir

4.1 The Transform

Definition 9 (Fiat–Shamir). Given a Sigma protocol and a hash function H with range $\mathcal{C}$, replace the verifier’s challenge by

$$c = H(x \parallel a \parallel \text{ctx}).$$

The proof is (a, z) , or (c, z) with a recomputed from the verification equation.

The prover now runs alone. The result is **non-interactive** (a proof is a string), **publicly verifiable** (anyone can check it), and, with a message placed in ctx , a **signature**. ML-DSA (Lecture 18) is exactly this construction, for a Sigma protocol over module lattices.

4.2 The Random Oracle Model

Definition 10 (Random oracle model). H is modelled as a uniformly random function, reachable only by queries. A security proof may see every query and choose the answers.

Both properties are used. *Extraction*: find the query that hashed a and rerun the prover with a different answer, which is rewinding on the oracle (the forking lemma). *Simulation*: run the Sigma simulator to get (a, c, z) , then define $H(x \parallel a) := c$. No concrete hash is a random oracle, so the proof is about an idealisation.

4.3 What Fiat–Shamir Costs

Grinding. A cheating prover tries many a and keeps one whose challenge it can answer. With Q hash queries its success probability is about $Q/|\mathcal{C}|$; for $|\mathcal{C}| = 2^{128}$ and $Q = 2^{80}$ this is 2^{-48} , not 2^{-128} .

Deniability. An interactive transcript convinces no third party, since anyone can simulate one (Example 2). A Fiat–Shamir proof is transferable.

Not lost: zero knowledge. The programmed-oracle simulator is as good as the interactive one.

Remark (Weak Fiat–Shamir). Hashing only a , without the statement x , lets an attacker fix the transcript first and then pick a statement that fits it. Hash the statement, the commitment, the public parameters and a domain separator.

5. Arithmetisation

Schnorr proves knowledge of one discrete logarithm. To prove “*I ran this program and got that answer*”, the witness is an execution trace of N steps, and the claim is turned into a polynomial identity.

5.1 From a Computation to a Polynomial

1. **Index steps by a subgroup.** Let $H = \{1, \omega, \dots, \omega^{N-1}\} \subseteq \mathbb{F}_p^\times$ with ω a primitive N -th root of unity (Lecture 2). Step i is ω^i , and “next step” is $X \mapsto \omega X$.
2. **Interpolate the trace.** Each register column becomes a polynomial $T(X)$ of degree $< N$, computed by an inverse NTT.

3. **Write the rules as a constraint.** One polynomial $C(T(X), T(\omega X), \dots)$ such that the computation is correct if and only if $C(h) = 0$ for every $h \in H$.

Proposition 6 (Lecture 2). $Z_H(X) = \prod_{h \in H} (X - h) = X^N - 1$, and

$$C(h) = 0 \text{ for all } h \in H \iff Z_H \mid C \iff C(X) = Q(X) Z_H(X) \text{ for some polynomial } Q.$$

On a subgroup Z_H has two terms, so the verifier evaluates it in $O(\log N)$. An honest prover's Q exists; for a dishonest prover no polynomial Q satisfies the identity.

5.2 The Protocol

Prover	Verifier
interpolates T , forms C	—
divides: $Q = C/Z_H$	—
commits to T and Q	—
—	samples $r \leftarrow \mathbb{F} \setminus H$
opens $T(r), T(\omega r), Q(r)$	recomputes $C(r)$
—	checks $C(r) = Q(r)(r^N - 1)$

A claim about N steps becomes one field identity, checked once. The prover must be bound to Q before it sees r ; otherwise it reports $Q(r) := C(r)/Z_H(r)$. Enforcing this is the job of a polynomial commitment (Section 6).

Proposition 7 (Soundness of the single check). *Let $d = \max(\deg C, \deg Q + N)$ and let r be uniform on $S \subseteq \mathbb{F}$. If $Z_H \nmid C$, then*

$$\Pr_r [C(r) = Q(r)Z_H(r)] \leq \frac{d}{|S|}.$$

Proof. $C - QZ_H$ is a nonzero polynomial of degree at most d ; apply Schwartz–Zippel (Lecture 2). $\square$

If $\mathbb{F}$ is too small for $d/|S|$ to be negligible, sample r from an **extension field** $\mathbb{F}_{p^k}$, or **repeat** the check at independent points.

5.3 Worked Instances

Example 4 (Arithmetise four steps). Work in $\mathbb{F}_{17}$ with $\omega = 13$, of order 4: $13^2 = 169 \equiv 16$, $13^3 \equiv 4$, $13^4 \equiv 1$. So $H = \{1, 13, 16, 4\}$ and $Z_H = X^4 - 1$. One register, one rule: *it always holds a bit*. The claimed trace is $(1, 0, 1, 0)$.

Interpolate. $T(X) = 9 + 9X^2$, using $2^{-1} \equiv 9$. Check: $T(1) = 18 \equiv 1$; $13^2 \equiv -1$ so $T(13) = 0$; $16^2 \equiv 1$ so $T(16) = 1$; $4^2 \equiv -1$ so $T(4) = 0$. $\checkmark$

Constrain. The rule is $T(T - 1) = 0$, so $C = T^2 - T$. With $81 \equiv 13$, $T^2 = 13 + 9X^2 + 13X^4$ (the X^2 coefficient is $2 \cdot 81 \equiv 9$), and

$$C(X) = T^2 - T = 4 + 13X^4.$$

Divide. $13X^4 + 4 = 13(X^4 - 1) + 17 \equiv 13(X^4 - 1)$, so $Q(X) = 13$, a constant.

Example 5 (The verifier’s single check). Same setting.

Honest prover. At $r = 3$: $3^4 = 81 \equiv 13$, so $C(3) = 4 + 13 \cdot 13 = 173 \equiv 3$ and $Q(3)Z_H(3) = 13 \cdot 12 = 156 \equiv 3$. ✓ At $r = 5$: $5^4 = 625 \equiv 13$, so again both sides are 3. ✓ At $r = 16$: $16^4 \equiv 1$, so $Z_H(16) = 0$ and $C(16) = 0$; both sides are 0. This r is a bad choice: $16 \in H$, so the check reduces to $C(16) = 0$ and says nothing about Q . This is why r is drawn from $\mathbb{F} \setminus H$.

Cheating prover. The claimed trace is $(1, 0, 1, 2)$. Interpolating gives $\tilde{T} = 1 + 15X + 2X^3$, and $\tilde{C} = \tilde{T}^2 - \tilde{T} = 15X + 4X^2 + 2X^3 + 9X^4 + 4X^6$ takes the values $(0, 0, 0, 2)$ on $(1, 13, 16, 4)$. Since $\tilde{C}(4) \neq 0$, $Z_H \nmid \tilde{C}$ and no Q exists. Whatever Q of degree ≤ 2 the prover commits to, $\tilde{C} - QZ_H$ is nonzero of degree at most 6, so with $S = \mathbb{F}_{17} \setminus H$ the error is at most $6/13$.

Example 6 (How big must the field be?). Take $N = 2^{20}$ steps with degree-3 constraints, so $d = 3 \cdot 2^{20} = 3\,145\,728$.

Field	Size	$d/ \mathbb{F} $
ML-DSA’s prime $q = 8\,380\,417$	$\approx 2^{23}$	$0.375 \approx 2^{-1.4}$
its degree-3 extension	$\approx 2^{69}$	$2^{-47.4}$
its degree-5 extension	$\approx 2^{115}$	$2^{-93.4}$
a 64-bit prime	2^{64}	$2^{-42.4}$
a 128-bit prime	2^{128}	$2^{-106.4}$

The prime field alone accepts a false proof with probability 37.5%. The degree-5 extension and the 128-bit prime are usable for one check; the others need repetition (for example, two independent checks over the 64-bit prime give $2^{-84.8}$).

6. Polynomial Commitments

6.1 The Interface

Definition 11 (Polynomial commitment scheme). Four algorithms: $\text{Setup}(d)$; $\text{Commit}(f) \rightarrow \text{cm}$; $\text{Open}(f, r) \rightarrow (y, \pi)$, claiming $f(r) = y$; $\text{Verify}(\text{cm}, r, y, \pi) \rightarrow \{0, 1\}$. It is **binding** if no efficient prover opens cm to two values at one point, and **succinct** if cm and π are much smaller than f .

Arithmetisation is information-theoretic; the commitment is the cryptographic part. Every assumption of a modern proof system lives in the polynomial commitment.

6.2 KZG

Setup. A trusted party samples a secret τ , publishes the structured reference string

$$\text{srs} = (g_1, g_1^\tau, g_1^{\tau^2}, \dots, g_1^{\tau^d}; g_2, g_2^\tau),$$

and destroys τ .

Commit. For $f(X) = \sum_{i=0}^d a_i X^i$,

$$\text{cm} = g_1^{f(\tau)} = \prod_{i=0}^d (g_1^{\tau^i})^{a_i},$$

computed without knowing τ . It is one group element, whatever d is.

Open. To claim $f(r) = y$: r is a root of $f(X) - y$, so the division is exact,

$$q(X) = \frac{f(X) - y}{X - r}, \quad \pi = g_1^{q(\tau)}.$$

Verify with two pairings (Lecture 9):

$$e(\text{cm} \cdot g_1^{-y}, g_2) = e(\pi, g_2^\tau \cdot g_2^{-r}).$$

Proposition 8. *An honest opening verifies.*

Proof. By bilinearity the left side is $e(g_1, g_2)^{f(\tau)-y}$ and the right side is $e(g_1, g_2)^{q(\tau)(\tau-r)}$; these agree because $f(X) - y = q(X)(X - r)$ holds identically, hence at $X = \tau$. $\square$

A wrong value y' leaves a nonzero remainder, so $(f(X) - y')/(X - r)$ is not a polynomial and the srs cannot express g_1 raised to it.

Example 7 (The KZG quotient). Over $\mathbb{F}_{17}$ let $f(X) = 3 + 2X + 5X^2 + X^3$ and $r = 2$. Then $y = f(2) = 3 + 4 + 20 + 8 = 35 \equiv 1$. Divide $f(X) - 1 = X^3 + 5X^2 + 2X + 2$ by $X - 2$ synthetically: the coefficients 1, 5, 2, 2 give 1; $5 + 2 = 7$; $2 + 14 = 16$; $2 + 32 = 34 \equiv 0$. So

$$q(X) = X^2 + 7X + 16, \quad \text{remainder } 0.$$

Check: $(X^2 + 7X + 16)(X - 2) + 1 = X^3 + 5X^2 + 2X - 31$, and $-31 \equiv 3$. $\checkmark$

False claim $y' = 5$: the same division of $f(X) - 5$ leaves remainder $y - y' = 1 - 5 \equiv 13 \neq 0$. No polynomial quotient exists, so the prover cannot form π from the srs.

Verification. Both sides are $e(g_1, g_2)$ to the exponents $f(\tau) - 1$ and $q(\tau)(\tau - 2)$, equal because $f(X) - 1 = q(X)(X - 2)$.

The trusted setup. Whoever knows τ computes $(f(\tau) - y')/(\tau - r)$ as a field element and opens any commitment to any value, and the opening verifies. The mitigation is a multi-party ceremony with $\tau = \tau_1 \tau_2 \cdots \tau_n$, safe if one participant is honest. KZG is not post-quantum: Shor's algorithm recovers τ from the public g_1^τ . Soundness fails; zero knowledge survives.

6.3 Reed–Solomon Codes

Definition 12 (Reed–Solomon code). For a domain $D \subseteq \mathbb{F}_p$ with $|D| = M$ and a degree bound $k < M$,

$$\text{RS}[D, k] = \{ (a(x))_{x \in D} : \deg a < k \} \subseteq \mathbb{F}_p^M.$$

Its **rate** is k/M and its **blow-up** is M/k .

A codeword is a polynomial in evaluation form.

Proposition 9 (Minimum distance). *Distinct codewords of $\text{RS}[D, k]$ differ in at least $M - k + 1$ positions.*

Proof. For $a \neq b$, $a - b$ is nonzero of degree $< k$, so it has at most $k - 1$ roots in D . $\square$

Example 8 (Reed–Solomon over $\mathbb{F}_{17}$). $\mathbb{F}_{17}^\times$ is cyclic of order 16, so it has one subgroup of order 8, the subgroup of index 2, which is the set of nonzero squares: $D = \{1, 2, 4, 8, 9, 13, 15, 16\}$. For $k = 3$: rate $3/8$, blow-up $8/3$, minimum distance $8 - 3 + 1 = 6$.

Encode $a(X) = 1 + 2X + 3X^2$; for instance $a(4) = 57 \equiv 6$ and $a(9) = 262 \equiv 7$. A second codeword agreeing with a at exactly two points is $b(X) = a(X) + (X - 1)(X - 2) = 3 + 16X + 4X^2$:

$x \in D$	1	2	4	8	9	13	15	16
$a(x)$	6	0	6	5	7	7	9	2
$b(x)$	6	0	12	13	12	3	4	8

They agree only at the roots $x = 1, 2$ of $b - a$, so their distance is 6, meeting the bound. ✓

6.4 FRI

Commit. Evaluate f (degree $< d$) on a subgroup D with $|D| = d/\rho$, for a rate ρ of $1/4$ or $1/8$, and build a Merkle tree over the evaluations. The commitment is the root. A root commits to *any* vector; FRI proves that the vector is close to a codeword of $\text{RS}[D, d]$.

Fold. Split $f(X) = f_e(X^2) + Xf_o(X^2)$. The verifier sends a random α , and the prover commits to

$$f'(Y) = f_e(Y) + \alpha f_o(Y), \quad \deg f' < d/2,$$

on $D' = \{x^2 : x \in D\}$, half the size. Repeated folding needs $|D|$ to be a power of two dividing $p - 1$.

Check the fold. For $x \in D$, also $-x \in D$ and $(-x)^2 = x^2$. From $f(\pm x) = f_e(x^2) \pm x f_o(x^2)$,

$$f_e(x^2) = \frac{f(x) + f(-x)}{2}, \quad f_o(x^2) = \frac{f(x) - f(-x)}{2x},$$

so $f'(x^2)$ follows from two evaluations of f .

1. **Commit phase.** $\log_2 d$ folds; the last polynomial is a constant, sent in the clear.
2. **Query phase.** s times: pick a random $x \in D$, open $f_i(\pm x_i)$ and $f_{i+1}(x_i^2)$ with Merkle paths at every layer, and check each fold.

The only assumption is a collision-resistant hash.

Example 9 (One FRI fold). Over $\mathbb{F}_{17}$ let $D = \langle 9 \rangle = \{1, 9, 13, 15, 16, 8, 4, 2\}$, of order 8. Since $9^4 = 6561 \equiv 16 = -1$, $9^{i+4} = -9^i$, so D splits into the pairs $\pm 1 = \{1, 16\}$, $\pm 9 = \{9, 8\}$, $\pm 13 = \{13, 4\}$, $\pm 15 = \{15, 2\}$. Take $f(X) = 3 + 2X + 5X^2 + X^3$, of degree < 4 (rate $1/2$).

Split and fold. $f_e(Y) = 3 + 5Y$, $f_o(Y) = 2 + Y$. With $\alpha = 4$,

$$f'(Y) = (3 + 4 \cdot 2) + (5 + 4 \cdot 1)Y = 11 + 9Y$$

on $D' = \{1, 13, 16, 4\}$.

Check at every pair, with $2^{-1} \equiv 9$:

x	$-x$	x^2	$f(x)$	$f(-x)$	$f_e(x^2)$	$f_o(x^2)$	$f_e + 4f_o$	$11 + 9x^2$
1	16	1	11	5	8	3	3	3
9	8	13	16	1	0	15	9	9
13	4	16	11	2	15	1	2	2
15	2	4	11	1	6	6	13	13

For instance at $x = 9$: $f_e = (16 + 1)/2 = 0$ and $f_o = 15/18 \equiv 15$ (since $18 \equiv 1$), so $f' = 0 + 4 \cdot 15 = 60 \equiv 9$. ✓

Fold again with $\beta = 5$: $f'' = 11 + 5 \cdot 9 = 56 \equiv 5$, a constant, on $\{1, 16\}$. Two folds for degree < 4 , as $\log_2 4 = 2$.

6.5 Why Spot-Checks Suffice

Proposition 10 (Query bound). *If a word is at distance at least δM from the code, and a test fails at every position where the word departs from it, then s independent uniform queries all miss with probability at most $(1 - \delta)^s$.*

Proof. Each query lands outside the at least δM failing positions with probability at most $1 - \delta$, independently. $\square$

Example 10 (Sizing the queries). For $M = 1024$, $k = 256$ the distance is 769; for a word that far away, 20 queries all miss with probability $(255/1024)^{20} \approx 8.4 \times 10^{-13} \approx 2^{-40.1}$.

FRI certifies much less than the minimum distance. At $\rho = 1/4$ take conservatively $\delta = 1/4$, so the error is 0.75^s and $\log_2 0.75 \approx -0.415$:

$$s = 80 \Rightarrow 2^{-33.2}, \quad s = 241 \Rightarrow 2^{-100.0}, \quad s = 309 \Rightarrow 2^{-128.2}.$$

Hundreds of queries, each with Merkle paths at every layer: STARK proofs are kilobytes.

7. SNARKs and STARKs

SNARK	Succinct Non-interactive ARGument of Knowledge
STARK	Scalable Transparent ARGument of Knowledge

Transparent means no trusted setup. Zero knowledge is in neither name: it is added by making the commitments hiding. Both families arithmetise identically (Section 5); only the polynomial commitment differs. KZG gives a SNARK, FRI gives a STARK. KZG tests the identity at a hidden point τ ; FRI has no hidden point, so it must sample, and transparency is paid for in queries. In production, a computation is often proved with a STARK and the STARK verifier is then wrapped in a SNARK, which is no longer post-quantum.

	SNARK (KZG)	STARK (FRI)
Setup	trusted	transparent
Assumption	DLog in a pairing group, ROM	collision-resistant hash, ROM
Proof size	≈ 200 bytes, constant	tens to hundreds of KB
Verifier	a few pairings (two per opening), $O(1)$	$O(\log^2 n)$ hashes
Prover	$O(n \log n)$, group operations	$O(n \log n)$, NTTs and hashes
Field	curve scalar field, $\approx 2^{254}$	two-adic prime, 2^{31} or 2^{64}
Post-quantum	no	yes

A SNARK gives tiny proofs with a trusted setup and is broken by Shor; a STARK is transparent and post-quantum, with proofs about 1000 times larger.

Summary

1. A proof system is complete, sound and zero knowledge. Zero knowledge means a simulator; knowledge means an extractor.
2. Schnorr: $a = g^k$, $z = k + cx$, check $g^z = ah^c$. Extract $x = (z - z')(c - c')^{-1}$; simulate with $a = g^z h^{-c}$.
3. Pedersen $g^m h^r$ is perfectly hiding, DLog-binding and additively homomorphic. Fiat–Shamir sets $c = H(x||a||\text{ctx})$.
4. Arithmetisation: the computation is correct iff $C = Q Z_H$, checked at one random r with error $d/|S|$.
5. The polynomial commitment decides the system: KZG (pairings, trusted setup) gives a SNARK; Merkle trees with FRI (hashes, Reed–Solomon) give a post-quantum STARK.

Exercises

1. In the group of Example 1, run Schnorr’s protocol with $k = 3$ and $c = 4$, and check the verification equation. Rewind with $c' = 10$, compute z' , and run the extractor to recover $x = 6$.
2. With $g = 2$, $h = 13$ as in Example 3, compute $\text{Com}(7; 2)$ and $\text{Com}(2; 8)$, and verify the homomorphism on them. If the committer had chosen $h = g^t$ with t known to it, which property would the scheme lose, and how would it open a commitment two ways?
3. A protocol has challenge space $\mathcal{C}$. (a) With $|\mathcal{C}| = 2$, how many parallel repetitions give interactive soundness error below 2^{-128} ? (b) After Fiat–Shamir, against an attacker making $Q = 2^{80}$ hash queries, how large must $|\mathcal{C}|$ be for error 2^{-128} ?
4. Show that the simulator of Section 2.2 fails against a verifier that sets $c = H(a)$, and explain why this does not contradict HVZK.
5. In the setting of Example 7, open f at $r = 3$: compute $y = f(3)$ and the quotient $q(X)$, and check $q(X)(X - 3) + y = f(X)$.
6. Repeat Example 9 with $\alpha = 6$. Verify the fold identity at $x = 9$ and $x = 15$, then fold again with $\beta = 2$ and give the final constant.
7. Over $\mathbb{F}_{17}$ with D and $k = 3$ as in Example 8, encode $c(X) = 5 + X + X^2$, and build a second codeword that agrees with it exactly at $x = 4$ and $x = 9$. Confirm the two are at distance 6.